System Architecture
Music Swipe Discovery App

Purpose. This version shows the major system components, the main API routes, the database structure, and the end-to-end data flow. It is detailed enough for planning and documentation now, while also including Firebase Authentication, liked and passed song tracking, Spotify-based song retrieval, and personalized recommendation logic.

Architecture Overview

Layer	Main Responsibility	Key Elements	Outputs
User / Client	Interacts with the app through a browser	Login, swipe actions, account view, liked/passed history	Likes, passes, auth state, view requests
React Frontend	Displays tracks and sends requests to backend	AuthScreen, Header, CardStack, SongCard, AccountView, fetch requests	Rendered swipe cards, account stats, user actions
Node / Express Backend	Central logic and routing layer	GET /songs, POST /like, POST /pass, GET /likedSongs/:uid, GET /passedSongs/:uid	Spotify results, recommendation results, Firestore writes, JSON responses
External Services	Provide music data and persistent storage	Spotify Web API, Firebase Auth, Firebase Firestore	Track metadata, authenticated users, stored liked/passed songs

Detailed Data Flow

1. Song retrieval — The React frontend requests songs from the backend using GET /songs?q=... . The backend requests an access token from Spotify, calls the Spotify search endpoint, reformats the response, and returns clean track objects to the frontend.

2. Card rendering — The frontend converts backend song objects into the format expected by the card UI and displays them as swipeable cards, including song title, artist, album art, and preview information if available.

3. Like and pass actions— When a user swipes right, the frontend sends POST /like with the current user id and song metadata. When a user swipes left, it sends POST /pass. These actions update both the interface and the user's stored history.

4. Song persistence — The backend writes the liked song into Firestore under users/{uid}/likedSongs/{spotifyId}, which prevents duplicates and keeps each user's liked songs grouped together.

5. Playlist retrieval — When the account view is opened, the frontend calls GET /likedSongs/:uid. The backend reads the user's liked songs from Firestore and returns them as JSON for display in the user's activity history.

6. Personalized recommendation flow — The backend reads the user's liked and passed songs, builds a taste profile using artists, albums, and title keywords, generates Spotify search queries, scores candidate tracks, filters out songs the user already liked or passed, and returns the top recommended tracks.

7. Account activity retrieval — When the user opens the account view, the frontend calls the liked/passed song and count routes to display saved songs, skipped songs, like rate, pass rate, and total decisions.

Current Backend Routes

Route	Method	Purpose	Used by
/	GET	Simple health check to confirm the backend is running.	Developer / browser
/test-db	GET	Verifies that Firestore is connected and readable.	Developer testing
/songs?q=...	GET	Fetches tracks from Spotify using direct search or personalized recommendation logic based on liked and passed songs.	Swipe UI
/like	POST	Stores a liked song in Firestore for the current user.	Swipe UI

/likedSongs/:uid	GET	Returns all liked songs for the selected user.	Playlist view
/pass	POST	Stores a passed song in Firestore for the current user.	Swipe UI
/passedSongs/:uid	GET	Returns all passed songs for the selected user.	Account view
/likedCount/:uid	GET	Returns number of liked songs.	Header / Account view
/passedCount/:uid	GET	Returns number of passed songs.	Header / Account view
/preview?song=...&artist=...	GET	Attempts to retrieve a song preview URL	Song card / preview support

Firestore Structure

```
users/{uid}
      email
      displayName
      createdAt

users/{uid}/likedSongs/{spotifyId}
      spotifyId
      title
      artist
      album
      imageUrl
      previewUrl
      likedAt
users/{uid}/passedSongs/{spotifyId}
```

spotifyId

title

artist

album

imageUrl

previewUrl

passedAt

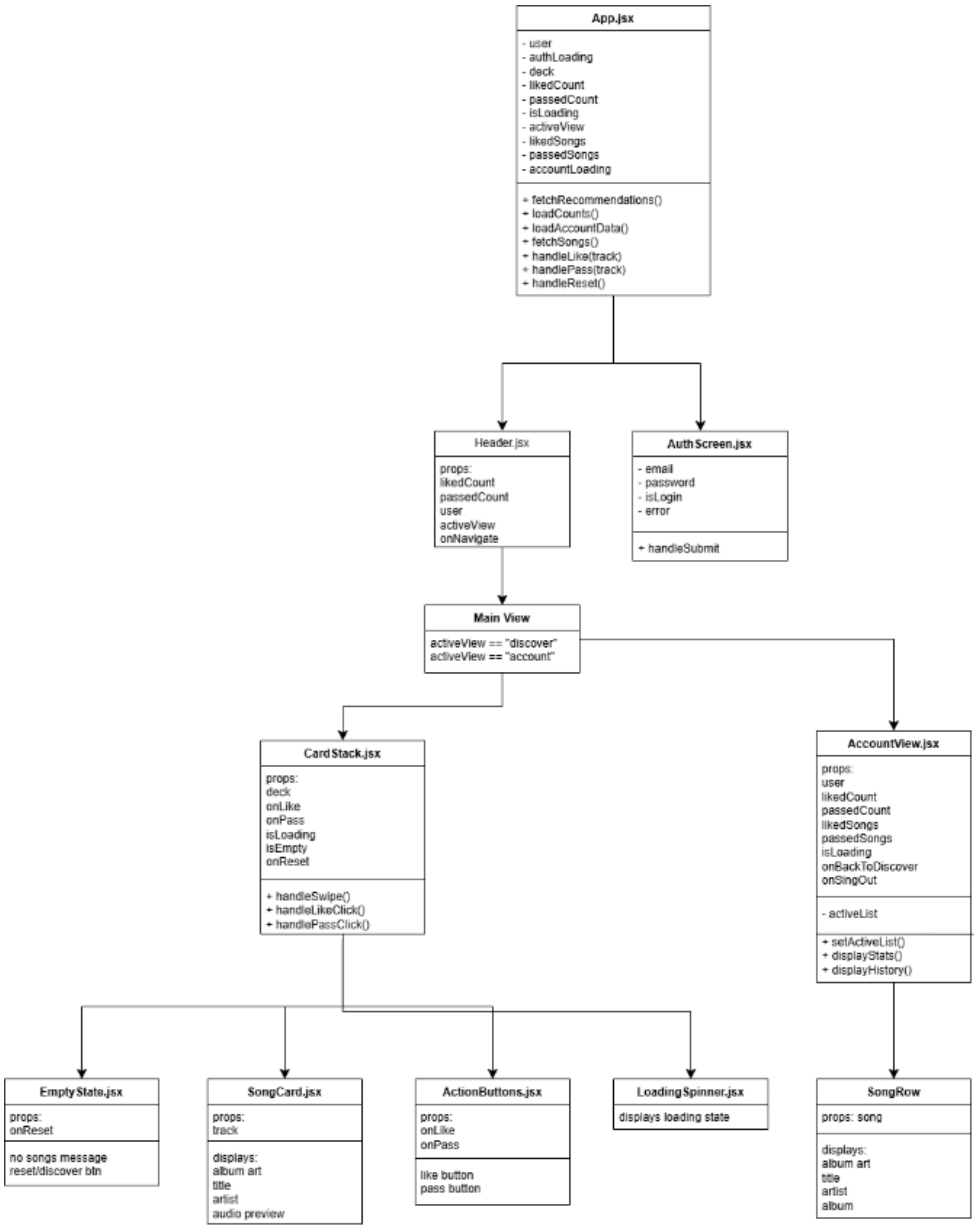
Firebase Authentication

uid

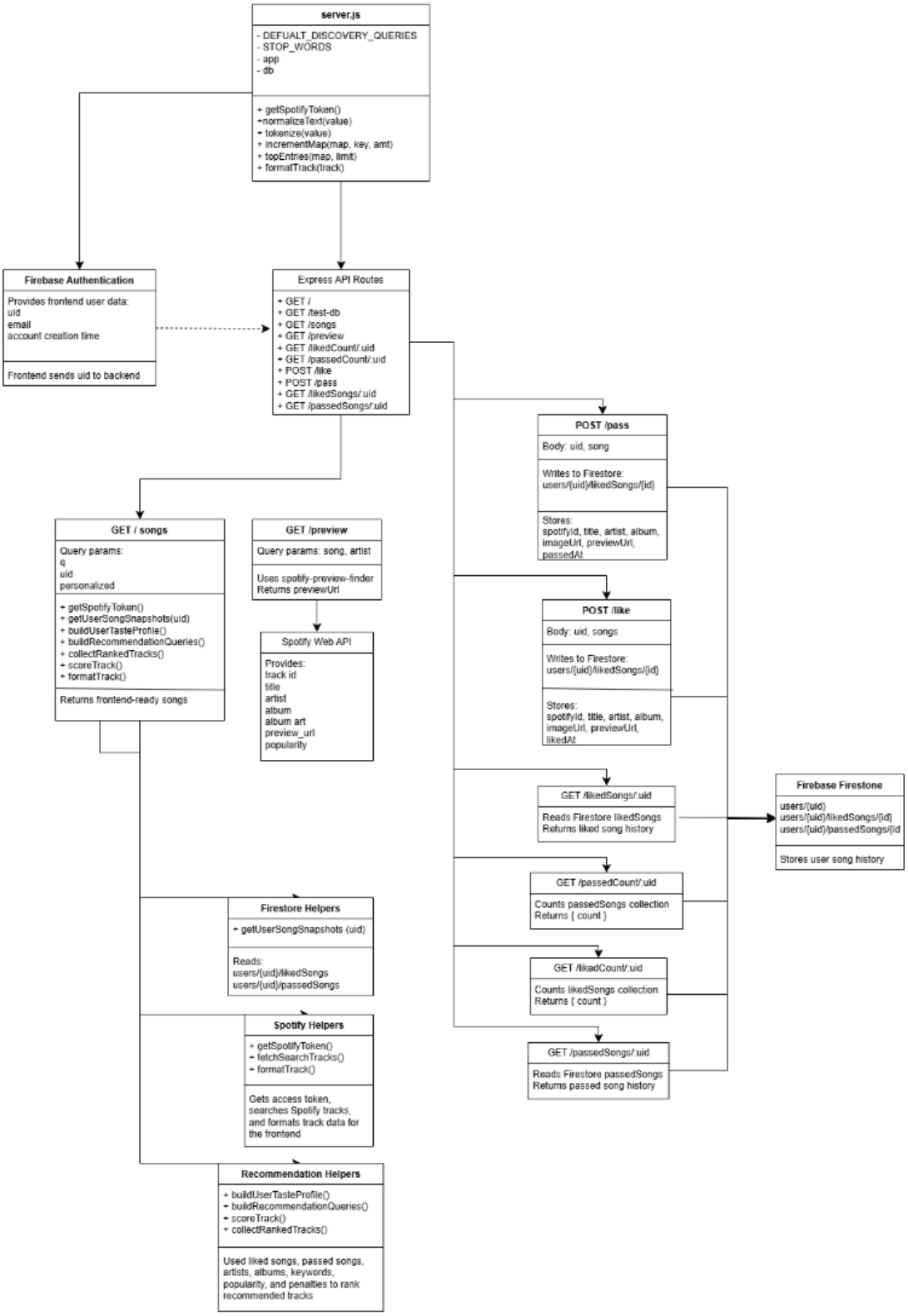
email

creationTime

Front end UML



Back end UML


server.js
- DEFUALT_DISCOVERY_QUERIES
- STOP_WORDS
- app
- db
+ getSpotifyToken()
+normalizeText(value)
+ tokenize(value)
+ incrementMap(map, key, amt)
+ topEntries(map, limit)
+ formatTrack(track)
Firebase Authentication
Provides frontend user data:
uid
email
account creation time
Frontend sends uid to backend
Express API Routes
+ GET /
+ GET /test-db
+ GET /songs
+ GET /preview
+ GET /likedCount/:uid
+ GET /passedCount/:uid
+ POST /like
+ POST /pass
+ GET /likedSongs/:uid
+ GET /passedSongs/:uid
GET / songs
Query params:
q
uid
personalized
+ getSpotifyToken()
+ getUserSongSnapshots(uid)
+ buildUserTasteProfile()
+ buildRecommendationQueries()
+ collectRankedTracks()
+ scoreTrack()
+ formatTrack()
Returns frontend-ready songs
GET /preview
Query params: song, artist
Uses spotify-preview-finder
Returns previewUrl
Spotify Web API
Provides:
track id
title
artist
album
album art
preview_url
popularity
POST /pass
Body: uid, song
Writes to Firestore:
users/{uid}/likedSongs/{id}
Stores:
spotifyId, title, artist, album, imageUrl, previewUrl, passedAt
POST /like
Body: uid, songs
Writes to Firestore:
users/{uid}/likedSongs/{id}
Stores:
spotifyId, title, artist, album, imageUrl, previewUrl, likedAt
GET /likedSongs/:uid
Reads Firestore likedSongs
Returns liked song history
Firebase Firestone
users/{uid}
users/{uid}/likedSongs/{id}
users/{uid}/passedSongs/{id
Stores user song history
GET /passedCount/:uid
Counts passedSongs collection
Returns { count }
GET /likedCount/:uid
Counts likedSongs collection
Returns { count }
GET /passedSongs/:uid
Reads Firestore passedSongs
Returns passed song history
Firestore Helpers
+ getUserSongSnapshots (uid)
Reads:
users/{uid}/likedSongs
users/{uid}/passedSongs
Spotify Helpers
+ getSpotifyToken()
+ fetchSearchTracks()
+ formatTrack()
Gets access token, searches Spotify tracks, and formats track data for the frontend
Recommendation Helpers
+ buildUserTasteProfile()
+ buildRecommendationQueries()
+ scoreTrack()
+ collectRankedTracks()
Used liked songs, passed songs, artists, albums, keywords, popularity, and penalties to rank recommended tracks

Implemented Additions Since Version 1

- Firebase Authentication for login, logout, and user-specific data.
- Passed song tracking to improve recommendations and avoid repeated skipped songs.
- Personalized recommendation scoring based on liked artists, liked albums, title keywords, disliked artists, and disliked keywords.
- Account dashboard showing liked count, passed count, total decisions, like rate, pass rate, and song history.

Possible Future Additions

- Saved named playlists.
- Music match feature between users.
- Deployment monitoring and better error handling.
- More advanced genre-based recommendation logic.

Final System summary: React frontend → Firebase Authentication → Node/Express backend → Spotify API + Firebase Firestore